

IOS Interface Documentation

1. Purpose

The *TPLSDK iOS SDK Interface Documentation* describes how third-party apps can use the iOS SDK for secondary development, enabling network configuration control and printing functionalities for label printers.

2. Application Scenarios

Supported iOS version: iOS 13.0 and above

Xcode version: 15.0 and above

3. Supported Device Types

- 1. Device Chip Types: Feiyitong (FYT) | Jihai (JH) | Chip (Chip)

- Obtain chip type: `BlueToothKit.share.chipType`

```
public enum BlueDeviceChip {  
    case None  
    case FYT  
    case JH  
    case HF  
    case JL  
}
```

- 2. Obtain device type: `BlueToothKit.share.deviceType`

```
public enum BlueDeviceType : String, CaseIterable {  
    case None  
    case TD401  
    case TD402S  
    case TP110  
    case TP120  
    case TP210  
    case TP220  
    case TD403  
    case TD80  
    case TD401_ETH  
}
```

- `.isSmallChip` indicates support for extremely small-sized chips (currently only TP110).

3. Obtain device-related information (effective after connection) (Refer to the example project TplSdkTest/DeviceInfoController.swift for detailed usage)

1. Custom Command Types

```
public enum CodeCommandType {  
    /// Get BLE name  
    case bleName  
    /// Get printer status  
    case printStatus  
    /// Get print mileage  
    case printMileage  
    /// Get paper-out threshold  
    case getOutPaper  
    /// Get label gap  
    case getLabelGap  
    /// Get black mark gap
```

```

    case getBlackGap
    /// Get printer version
    case printVersion
    /// Initialize printer
    case initializationPrint
    /// Gap auto-learning
    case gapAutoLearn
    /// Black mark auto-learning
    case blackGapAutoLearn
    /// Paper-out auto-learning
    case outPaperAutoLearn
    /// Self-check
    case selfAutoCheck
    /// Get distance from print head to tear-off position
    case tearHeight
    /// Get printer chip information
    case getChip
    /// Get printer battery level
    case getPower
    /// Get cutter mode
    case getCutter
    /// Get printer model
    case getModel
}

```

2. APIs for obtaining device information

```

`DeviceInfoTool.share.sendCommand(type: .getChip)`
`DeviceInfoTool.share.sendCommand(type: .printVersion)`

```

3. Notification Refresh

1. Add a listener object (typically added in the controller's `viewDidLoad()` or UIView's `init()` methods)

```

`DeviceInfoTool.regist(self)`

```

2. Implement the protocol notification method (`DeviceInfoToolProtocol`). Example:

```

...
extension ViewController: DeviceInfoToolProtocol {
    /// Called when chip information is obtained
    func onDeviceModelUpdate(value: Any?) {

    }
    /// Called when the current printer version is obtained
    func onDeviceVersionUpdate(value: Any?) {

    }
}
...

```

3. Unregister the listener object when the page is destroyed

```

...
deinit {
    DeviceInfoTool.share.unRegist(self)
}
...

```

4. API Usage Step-by-Step Guide

0. Scan for Bluetooth Devices

```

Initialize the scan model: BlueToothScanConfig
// Scan duration

```

- 0. var scanTime: Int = 10
// Enable name filtering
- 0. var isFilterName: Bool = false
// Filter name
- 0. var filterName: String?
// Filter irrelevant devices
- 0. var isFilterIrrelevant: Bool = false
// Enable RSSI filtering
- 0. var isFilterRSSI: Bool = false
// Minimum RSSI
- 0. var minRSSI: Int = -100
// Maximum RSSI
- 0. var maxRSSI: Int = 0
// Enable logs
- 0. var isShowLog: Bool = false

```
let scanConfig = BluetoothScanConfig(isFilterName: false, filterName: "", isFilterIrrelevant: true, isShowLog: false)
BluetoothKit.share.scanBlueDevices(config: scanConfig) { [weak self] (scanDevice, scanStatus) in
    // Permission not enabled
    if scanStatus == .off {
        DispatchQueue.main.async {
            self?.showAlert(title: "Notice", message: "Bluetooth must be enabled for this feature.")
        }
        return
    }

    guard let scanDevice = scanDevice else { return }

    // Process scanned device
    guard scanStatus == .normal else { return }
    self?.pageViewModel.addDevice(model: scanDevice) { [weak self] in
        DispatchQueue.main.async {
            self?.updateUI()
        }
    }
}
```

- 0.
 - case .normal // Normal
 - case .off // Permission not enabled
 - case .fail // Scan failed
 - case .end // Scan completed

- 0. Connect to Device

- 0.
 - - .success // Connection successful
 - .fail // Connection failed
 - .disconnected // Disconnected
 - .off // Permission not enabled

```
BluetoothKit.share.connectBlue(model: model) { [weak self] (device, status) in
    guard let self = self else { return }
    // Connection failed
    if status == .fail {
        self.showAlert(title: "Notice", message: "Bluetooth connection failed.")
        self.updateUI()
        return
    }
    // Connection successful
    if device.isConnected {
```

```

        // Update the current connected device
        BluetoothKit.share.updateConnect(blueDevice: device)
        // Update the current device in the view model
        self.pageViewModel.updateUniqueID(device: device)
        // Query device information
        DeviceInfoTool.share.sendCommand(type: .getChip)
        // Get version information
        DeviceInfoTool.share.sendCommand(type: .printVersion, after: 0.1)
    }
    // Notify for updates (when external components need to update connection status)
    ProtocolDispatcher(ConnectStatusProtocol.self) {
        $0.onConnectStatusChanged(value: device.isConnected)
    }
    self.updateUI()
}

```

0. WiFi Connection

SocketClient().connect(to host: String, port: UInt16, completeClosure: VoidHandlerClosure?, failureClosure: VoidHandlerClosure?)

- host: IP address
- port: Port number
- completeClosure: Success callback
- failureClosure: Failure callback

0. Disconnect Device

0. disconnectedDevice: The parameter is the device model.

```

BluetoothKit.share.disconnectBlue(model: model) { [weak self] (device, status) in
    guard let self = self else { return }
    if status == .disconnected {
        BluetoothKit.share.updateDisconnect()
        completeClosure?()
    }
    self.updateUI()
}

```

0. Data Sending

0. Bluetooth sending:

- data: Data to send
- sepSize: Data packet size, maximum not exceeding 500.
The closure returns values (send, total, status)
- send: Size of data already sent
- total: Total size of data
- status: Status of Bluetooth data transmission

- .normal // Normal
- .sending // Sending in progress
- .error // Send error
- .finish // Send completed

0. WiFi sending (no progress return):

- data: Data to send
- after: Specified delay (in seconds) before callback.

5. Common Errors and Solutions

1. clang: error: SDK does not contain 'libarclite' at the path

**'/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/lib/arc/libarclite_iphoneos.a';
try increasing the minimum deployment target**

This type of error is caused by issues with Xcode 15, as its minimum support for libraries is iOS 12.0. To resolve this, modify the minimum compatible version for both the Pod libraries and the project itself accordingly.